the components should be present in a single package/code base for the page to be rendered. In our example, a client would retrieve this data by making a single REST call (GET, *api.company.com/products/productId*) in the application. A load balancer or a cluster may route the request to one of N application instances. The application then queries various database tables and returns the response to the client.

The advantage of this design is that the various functionalities like logging, audit trails, middleware, etc, are all hooked to one app wherein any changes, corrections and optimisations can be done easily. One more advantage is that shared memory is faster than inter-process communication (IPC).

In spite of the advantages, there are many concerns in this pattern. As components are tightly coupled it is difficult to isolate services for any purpose such as independent scaling or code maintainability. The components become much harder to understand in the long run as the code base grows in size and is not obvious when looking at a particular service or controller. Down the line, the application is sure to go out of control; so this is not a good idea.

Microservice architecture

This design uses small, modular units of code that can be deployed independent of the rest of the product's components. An application is a collection of loosely coupled services and each service is fine grained, thus improving modularity and making the application easier to understand, as well as faster to develop, test and deploy.

- Microservices components are modular, so each service can be built, updated and deployed independent of any other code.
- Each microservice is responsible for a specific purpose or task.
- Each service abstracts away implementation details, exposing only a well-documented interface; so APIs can be consumed in a consistent way, regardless of how exactly the service is built.

Figure 1(a): *Product details* pages of Amazon

Figure 1(b): *Product details* pages of Amazon

Figure 1(c): *Product details* pages of Amazon

Figure 2: The difference between the two architectures

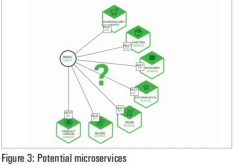

Figure 3: Potential microservices

In our shopping application, the data displayed on the 'Product details' page is from multiple microservices. Some of the potential microservices are:

- Shopping cart service
- Catalogue service
- Review service
- Shipping service
- Recommendation service

In theory, a client could make a request to each of the microservices, directly. Each service can have an end point like *http://service.api.company.com*. This URL would map to the microservice's load balancer, which distributes requests across the available instances. To retrieve the product details, the mobile client would make requests to each of the requests listed above.

Unfortunately, there are some challenges and difficulties with this option. One problem is the mismatch between the client and the fine grained APIs exposed. In this example, the client has to make seven separate requests for a page. In a complex application, hundreds of services may be involved in a page, and this would probably prove inefficient over a public network and quite impossible over a mobile network.

Another hurdle with the client directly calling the microservices is that it might use a protocol that may not be Web-friendly. One service may use RPC and another the AMQP messaging protocol. Neither of them is browser- or firewall-friendly, and both are best suited internally. An application should use only HTTP and Web Socket outside of the firewall.

Yet one more drawback with this approach of the client calling the microservices is that it is